

- TD 2 -

Représentation de graphes en Python

Ce qu'il faut savoir faire :

Savoir définir un graphe en Python et le représenter graphiquement :

- Exercice 1 - question 1,
- Exercice 2,
- Exercice 4 - questions 1, 2 et 3.

Savoir écrire quelques fonctions en Python relatives aux graphes :

- Exercice 3.

Pour ce TD, on commencera par importer les modules ci-dessous :

```
import numpy as np
import networkx as nx
```

Exercice 1 : *Le module Python networkx*

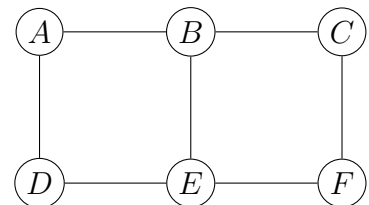
On pourra se reporter au document Python « Python 3 : le module networkx ».

1. Définir un graphe en Python

- (a) On considère le graphe non orienté ci-contre :
Les sommets sont ici définis comme des chaînes de caractères en Python.

Compléter la commande

```
G=nx.Graph(***)
# où *** est la liste des
# arêtes définissant
# le graphe ci-contre.
```



Vérifier votre script à l'aide des commandes :

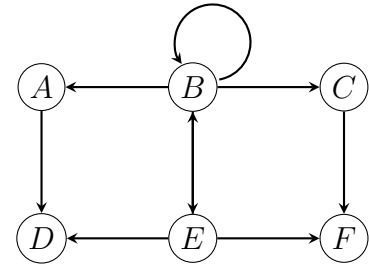
```
nx.draw(G, with_labels=True)
```

- (b) On considère le graphe orienté ci-contre.
Les sommets sont ici définis comme des chaînes de caractères en Python.
Compléter la commande

```
Go=nx.DiGraph(***)
# où *** est la liste des
# arcs définissant
# le graphe ci-contre.
```

Vérifier votre script à l'aide des commandes :

```
nx.draw(Go, with_labels=True)
```

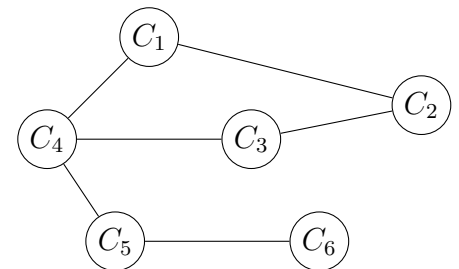


- (c) On reprend le graphe Gc ci-contre qui a été présenté en cours.

Déterminer la matrice d'adjacence M du graphe Gc.

Compléter les commandes ci-dessous,
puis les exécuter :

```
M=np.array([[...]])
Gc=nx.Graph(M)
nx.draw(Gc, with_labels=True)
```



Quelles commandes devez-vous ajouter pour obtenir le graphe ci-dessus, c'est-à-dire le graphe dont les sommets sont nommés $C_1, C_2, \dots, C_6$?

- (d) Reprendre les deux graphes des questions 1a et 1b et définir les graphes à l'aide d'un dictionnaire.

2. On rappelle ci-dessous quelques fonctions du module networkx : les tester sur les exemples précédents.

Quelques fonctions du module networkx

On suppose que G est une variable Python, définissant un graphe (orienté ou non).

La commande Python	renvoie :
<code>G.number_of_nodes()</code>	le nombre de sommets de G
<code>G.number_of_edges()</code>	le nombre d'arêtes (ou d'arcs) de G
<code>list(G.nodes())</code>	la liste des sommets de G
<code>list(G.edges())</code>	la liste des arêtes (ou arcs) de G
<code>list(G.adj[s])</code> où s est l'un des sommets de G	la liste des sommets adjacents (ou la liste des successeurs) du sommet s

Une option graphique

La commande Python	permet de disposer les sommets du graphes :
<code>position = nx.circular_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	en cercle
<code>position=nx.bipartite_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	sur deux rangées
<code>position=nx.multipartite_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	sur plusieurs rangées
<code>position=nx.bfs_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	selon l'algorithme de parcours en largeur
<code>position=nx.planar_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	sans arêtes (ou arcs) se croisant, lorsque c'est possible
<code>position=nx.random_layout(G)</code> <code>nx.draw(G,with_labels=True,pos=position)</code>	au hasard

Exercice 2 : Un exemple de graphe

Ecrire une fonction en Python, de paramètre `n` de type `int` représentant un entier n de $\mathbb{N}^*$, et qui renvoie la matrice d'adjacence du graphe orienté de sommets notés $1, 2, \dots, n$ et dont les arcs représentent la relation « est diviseur de », autrement dit, pour tout $(i, j) \in \llbracket 1, n \rrbracket^2$, j est un successeur de i si, et seulement si, i divise j .

Exercice 3 : Quelques fonctions Python sur les graphes

Dans cet exercice, on représentera un graphe par sa matrice d'adjacences, et on nommera les sommets d'un graphe d'ordre n , où $n \in \mathbb{N}^* : 0, 1, \dots, n-1$.

Dans toutes les questions qui suivent, M désigne un paramètre Python de type `array` qui représente la matrice d'adjacence d'un graphe orienté G à n sommets.

1. Représenter les graphes orientés associés aux matrices d'adjacences suivantes :

$$M_1 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \quad M_2 = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad M_3 = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}, \quad M_4 = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Dans la suite de l'exercice, vous pourrez tester la validité de vos fonctions en les testant sur certaines des quatre matrices d'adjacence ci-dessus (ou d'autres).

2. Ecrire une fonction Python de paramètre `M` qui renvoie le nombre n de sommets du graphe G .
On n'utilisera pas la commande `nx.number_of_nodes` du module `networkx`.
3. Ecrire une fonction Python de paramètre `M` qui renvoie le nombre de boucles du graphe G .
4. Ecrire une fonction Python de paramètres `M` et `i` de type `int`, qui renvoie
Non dans le graphe, si la valeur de `i` n'est pas l'un des sommets de G , et qui renvoie la liste des successeurs de `i` sinon.
5. Ecrire une fonction Python de paramètres `M` et `i` de type `int`, qui renvoie
Non dans le graphe, si la valeur de `i` n'est pas l'un des sommets de G , et qui renvoie le nombre des prédécesseurs de `i` sinon.
6. Ecrire une fonction Python de paramètres `M`, qui renvoie le nombre de sommets isolés de G .

Exercice 4 : Représentation des graphes complets

Soit $n \in \mathbb{N}^*$.

On note K_n le graphe complet à n sommets, numérotés de 1 à n .

1. Représenter les graphes complets K_2 , K_3 et K_4 .
2. Préciser le nombre d'arêtes du graphe K_n .
3. Ecrire la matrice d'adjacences M_n du graphe complet K_n , puis la définir en Python.
4. (a) Ecrire le dictionnaire définissant le graphe complet K_n .
(b) Ecrire une fonction Python de paramètre `D`, représentant le dictionnaire caractérisant un graphe G , et qui renvoie `True`, si le graphe G est complet, et `False` sinon.

Exercice 5 : Représentations équivalentes des graphes en Python

Vous pourrez tester la validité de vos fonctions en utilisant par exemple, les graphes présentés dans le polycopié de cours ou ceux des exercices 1 et 3.

1. Ecrire une fonction `test_mat`, de paramètre `M` de type `array`, qui renvoie :
 - **Non graphe**, si `M` n'est pas la matrice d'adjacence d'un graphe non valué,
 - **Graphe non orienté**, si `M` est la matrice d'adjacence d'un graphe simple non orienté,
 - **Graphe orienté**, sinon.
2. Ecrire une fonction Python, de paramètre `D` de type `dict` et représentant un graphe G à n sommets ($n \in \mathbb{N}^*$), qui renvoie un dictionnaire représentant le même graphe que G et dont les sommets sont nommés $0, 1, \dots, n-1$.
3. Ecrire une fonction Python nommée `mat_to_dict`, de paramètre `M` de type `array` représentant la matrice d'adjacences d'un graphe G , et qui renvoie un dictionnaire caractérisant G .
4. Ecrire une fonction Python nommée `dict_to_mat`, de paramètre `D` de type `dict` caractérisant un graphe G , et qui renvoie la matrice d'adjacences de G .
5. Lorsque `M` est une variable Python dont la valeur est la matrice d'adjacences d'un graphe G , a-t-on l'égalité `dict_to_mat(mat_to_dict(M))=M` ?
De même, si `D` est une variable Python dont la valeur est un dictionnaire définissant un graphe G , a-t-on l'égalité `mat_to_dict(dict_to_mat(D))=D` ?

Un sujet de concours Mines-Ponts sur les matrices de permutation et les matrices symétriques

Dans ce problème, on propose de définir la notion d'image d'une matrice réelle symétrique par une fonction d'une variable réelle, puis d'étudier quelques propriétés de cette notion (en particulier, relativement à la convexité). Ces notions présentent un intérêt en sciences physiques (statistique ou quantique).

Notations

Dans tout le problème :

- n désigne un entier naturel non nul ;
- si p et q sont des entiers naturels, l'ensemble des entiers k tels que $p \leq k \leq q$ est noté $\llbracket p, q \rrbracket$;
- si i et j sont des entiers naturels, alors $\delta_{i,j} = 1$ si $i = j$ et $\delta_{i,j} = 0$ sinon ;
- B_n désigne l'ensemble des bijections de $\llbracket 1, n \rrbracket$ dans lui-même ;
- I est un intervalle de $\mathbb{R}$ qui n'est ni vide ni réduit à un singleton ;
- $\mathcal{C}^0(I, \mathbb{R})$ désigne l'ensemble des fonctions continues de I dans $\mathbb{R}$;
- une fonction φ de I dans $\mathbb{R}$ est dite polynomiale s'il existe P un polynôme réel tel que, pour tout $x \in I$, $\varphi(x) = P(x)$;
- $\mathcal{M}_n(\mathbb{R})$ (respectivement $D_n(\mathbb{R})$, resp. $S_n(\mathbb{R})$, resp. $O_n(\mathbb{R})$), désigne l'ensemble des matrices carrées (resp. diagonales, resp. symétriques, resp. orthogonales) d'ordre n à coefficients réels, et on confond un élément de $\mathcal{M}_1(\mathbb{R})$ avec son unique coefficient ;
- on note Tr l'application trace définie sur $\mathcal{M}_n(\mathbb{R})$;
- si $M \in \mathcal{M}_n(\mathbb{R})$, on note $M^\top$ sa transposée, on note $\text{Sp}(M)$ son spectre réel, et si $(i, j) \in \llbracket 1, n \rrbracket^2$, $[M]_{i,j}$ est le coefficient de M situé à la i -ème ligne et j -ème colonne ;
- on munit $\mathcal{M}_n(\mathbb{R})$ de sa norme infinie, notée $\|\cdot\|$ et définie par :

$$\forall M \in \mathcal{M}_n(\mathbb{R}), \quad \|M\| = \max \{ |[M]_{i,j}|, 1 \leq i, j \leq n \}$$

- $S_n(I)$ désigne l'ensemble des matrices de $S_n(\mathbb{R})$ dont le spectre réel est inclus dans I ;
- si $u = (u_i)_{1 \leq i \leq n} \in \mathbb{R}^n$, on dit que ce n -uplet est croissant si pour tout $(i, j) \in \llbracket 1, n \rrbracket^2$,

$$(i \leq j) \implies (u_i \leq u_j)$$

- si $i_0 \in \llbracket 1, n \rrbracket$, on appelle nombre d'occurrences de u_{i_0} dans u le cardinal de l'ensemble $\{i \in \llbracket 1, n \rrbracket; u_i = u_{i_0}\}$
- enfin $\text{Diag}((u_i)_{1 \leq i \leq n})$ désigne l'élément D de $D_n(\mathbb{R})$ tel que :

$$\forall i \in \llbracket 1, n \rrbracket, [D]_{i,i} = u_i$$

on pourra noter cet élément en extension $D = \text{Diag}(u_1, \dots, u_n)$.

Partie I : Matrices de permutations

Le but de cette partie est d'étudier l'action sur les matrices diagonales de la conjugaison par des matrices de permutations. On considère l'application ω de B_n dans $\mathcal{M}_n(\mathbb{R})$ définie par :

$$\forall \sigma \in B_n, \forall (i, j) \in \llbracket 1, n \rrbracket^2, [\omega(\sigma)]_{i,j} = \delta_{i, \sigma(j)}$$

1. Démontrer que pour tout $(\sigma, \sigma') \in B_n^2$, $\omega(\sigma \circ \sigma') = \omega(\sigma)\omega(\sigma')$
2. Démontrer que $\omega(B_n) \subset O_n(\mathbb{R})$.
3. Soit $\sigma \in B_n$ et $(d_i)_{1 \leq i \leq n} \in \mathbb{R}^n$. Vérifier que :

$$\text{Diag}((d_i)_{1 \leq i \leq n}) \omega(\sigma) = \omega(\sigma) \text{Diag}((d_{\sigma(i)})_{1 \leq i \leq n})$$

4. En déduire l'équivalence suivante concernant deux éléments D et D' de $D_n(\mathbb{R})$,
 - (i) D et D' ont le même ensemble de coefficients diagonaux, chacun ayant le même nombre d'occurrences dans D et D' .
 - (ii) il existe $M \in \omega(B_n)$ telle que $D' = M^\top D M$.

Partie II : Fonctions de matrices symétriques

Cette partie a pour objectif de définir une correspondance entre l'espace des fonctions de I dans $\mathbb{R}$ et l'espace des fonctions de $S_n(I)$ dans $S_n(\mathbb{R})$, puis d'en démontrer quelques propriétés. Dans cette partie, f est une fonction de I dans $\mathbb{R}$.

5. Soit $S \in S_n(I)$. Justifier l'existence de $\Omega \in O_n(\mathbb{R})$ et de $(s_i)_{1 \leq i \leq n} \in I^n$ tels que :

$$S = \Omega^\top \text{Diag}((s_i)_{1 \leq i \leq n}) \Omega$$

6. Pour tout $(s_i)_{1 \leq i \leq n} \in I^n$, justifier l'existence d'un élément P de $\mathbb{R}[X]$ tel que :

$$\forall i \in \llbracket 1, n \rrbracket, \quad P(s_i) = f(s_i)$$

Soit $S \in S_n(I)$. On suppose que l'on dispose des deux écritures :

$$S = \Omega^\top \text{Diag}((s_i)_{1 \leq i \leq n}) \Omega \quad \text{et} \quad S = \Omega'^\top \text{Diag}((s'_i)_{1 \leq i \leq n}) \Omega'$$

avec $(\Omega, \Omega') \in O_n(\mathbb{R})^2$, $(s_i)_{1 \leq i \leq n} \in I^n$ et $(s'_i)_{1 \leq i \leq n} \in I^n$.

7. Montrer que l'on a alors :

$$\Omega'^\top \text{Diag}((f(s'_i))_{1 \leq i \leq n}) \Omega' = \Omega^\top \text{Diag}((f(s_i))_{1 \leq i \leq n}) \Omega$$

puis que $\Omega^\top \text{Diag}((f(s_i))_{1 \leq i \leq n}) \Omega \in S_n(\mathbb{R})$.

Dans la suite du problème, on note u l'application qui, à toute fonction φ de I dans $\mathbb{R}$, associe $u(\varphi)$ la fonction de $S_n(I)$ dans $S_n(\mathbb{R})$ définie par :

$$\forall S \in S_n(I), \quad u(\varphi)(S) = \Omega^\top \text{Diag}((\varphi(s_i))_{1 \leq i \leq n}) \Omega$$

où $S = \Omega^\top \text{Diag}((s_i)_{1 \leq i \leq n}) \Omega$, avec $\Omega \in O_n(\mathbb{R})$ et $(s_i)_{1 \leq i \leq n} \in I^n$.

Cette fonction est bien définie puisque, d'après la question précédente, $u(\varphi)(S)$ ne dépend pas du choix des matrices $\Omega \in O_n(\mathbb{R})$ et $D = \text{Diag}((s_i)_{1 \leq i \leq n})$ avec $(s_i)_{1 \leq i \leq n} \in I^n$, tel que $S = \Omega^\top D \Omega$.

Enfin, on désigne par v l'application $\text{Tr} \circ u$.

8. Vérifier que u et v sont linéaires, puis calculer, pour toute fonction φ de I dans $\mathbb{R}$ et pour tout $x \in I$, $u(\varphi)(xI_n)$.

9. Étudier l'injectivité et la surjectivité de u .
10. On suppose que f est polynomiale ; montrer qu'il existe $P \in \mathbb{R}[X]$ tel que pour tout $S \in S_n(I)$, $u(f)(S) = P(S)$.
Réciproquement, est-il vrai que, s'il existe $P \in \mathbb{R}[X]$ tel que pour tout $S \in S_n(I)$ $u(f)(S) = P(S)$, alors f est polynomiale ?
11. Démontrer que, si $(\varphi_k)_{k \in \mathbb{N}}$ est une suite de fonctions de I dans $\mathbb{R}$ qui converge simplement sur I vers une fonction φ , alors les suites $(u(\varphi_k))_{k \in \mathbb{N}}$ et $(v(\varphi_k))_{k \in \mathbb{N}}$ convergent simplement sur $S_n(I)$.
Y a-t-il convergence uniforme sur $S_n(I)$ si l'on suppose que $(\varphi_k)_{k \in \mathbb{N}}$ converge uniformément sur I ?

Partie III : Norme et convexité

L'objectif de cette partie est de munir $S_n(\mathbb{R})$ d'une nouvelle norme qui permettra de compléter l'étude des fonctions de matrices symétriques.

12. On note $\Sigma = \{X \in \mathcal{M}_{n,1}(\mathbb{R}); {}^tXX = 1\}$. Démontrer que si $S \in S_n(\mathbb{R})$ on a :

$$\min(\text{Sp}(S)) = \min \{ {}^tX S X; X \in \Sigma \} \quad \text{et} \quad \max(\text{Sp}(S)) = \max \{ {}^tX S X; X \in \Sigma \}$$

13. Montrer finalement que $S_n(I)$ est une partie convexe de $S_n(\mathbb{R})$ et que l'application ρ , de $S_n(\mathbb{R})$ dans $\mathbb{R}$, qui à toute matrice $M \in S_n(\mathbb{R})$ associe

$$\max\{|\lambda|; \lambda \in \text{Sp}(M)\}$$

est une norme sur $S_n(\mathbb{R})$.

Partie IV : Convexité des fonctions de matrices symétriques

On démontre maintenant quelques résultats relatifs à la convexité des fonctions de matrices symétriques. Dans cette partie, f est une fonction de I dans $\mathbb{R}$.

14. On suppose ici que f est convexe sur I et que $S \in S_n(I)$. On note

$$\mathcal{U}_S = \{ \Omega^\top S \Omega; \Omega \in O_n(\mathbb{R}) \}$$

Justifier que pour tout $U \in \mathcal{U}_S$, pour tout $k \in \llbracket 1, n \rrbracket$, $[U]_{k,k} \in I$. Démontrer alors que :

$$\max \left\{ \sum_{k=1}^n f([U]_{k,k}); U \in \mathcal{U}_S \right\} = v(f)(S)$$

15. En déduire que, si f est convexe sur I , pour tout $(A, B) \in S_n(I)^2$, pour tout $t \in [0, 1]$, on a :

$$v(f)((1-t)A + tB) \leq (1-t)v(f)(A) + tv(f)(B)$$

On dit qu'une fonction ψ de $S_n(I)$ dans $\mathbb{R}$ est convexe sur $S_n(I)$ si elle vérifie la relation :

$$\forall (A, B) \in S_n(I)^2, \forall t \in [0, 1], \quad \psi((1-t)A + tB) \leq (1-t)\psi(A) + t\psi(B)$$

16. Démontrer finalement que la fonction $v(f)$ est convexe sur $S_n(I)$ si, et seulement si, f est convexe sur I .